PROJECT: Real-Time Chat App using WebSockets and Serverless

Services: API Gateway (WebSocket API), Lambda, DynamoDB, Cognito
Goal: Create a scalable real-time messaging platform.
Skills: WebSocket management, identity federation, state handling.

Definition: A Real-Time Chat App using WebSockets and Serverless is an application that enables instant communication between users without page refresh. It uses Amazon API Gateway to maintain persistent connections for real-time data transfer. Backend processing is handled by AWS Lambda, which eliminates the need for managing servers. Messages and connection data are stored in Amazon DynamoDB for scalability and fast access. User authentication and identity management are managed using Amazon Cognito, making the system secure and scalable.

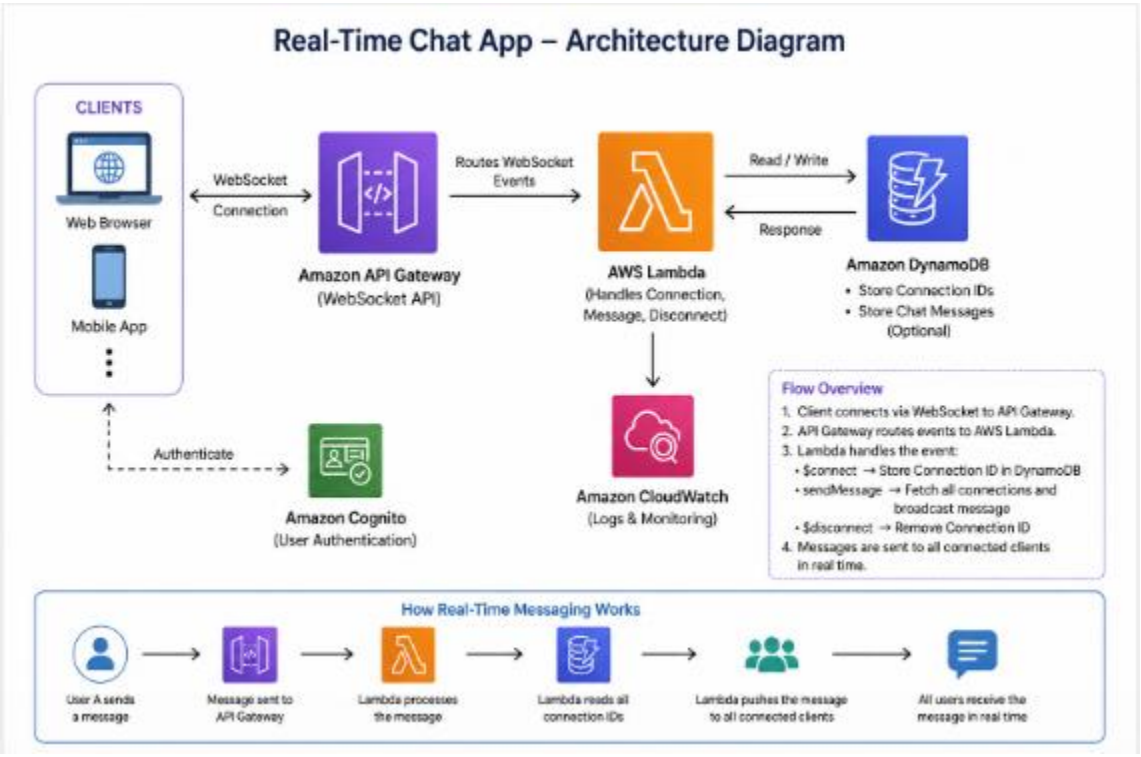
Scope of this project: The scope of a Real-Time Chat App using WebSockets and Serverless includes building a scalable messaging platform that supports instant communication between multiple users. It involves using Amazon API Gateway for real-time connections and AWS Lambda for backend processing without server management. The project can be extended to include user authentication with Amazon Cognito and data storage using Amazon DynamoDB. Additional features like group chats, message history, and notifications can be implemented. Overall, it demonstrates skills in serverless architecture, real-time systems, and cloud scalability.

Methodologies:
The project uses an event-driven architecture where messages are processed in real time through Amazon API Gateway. Backend operations are handled using AWS Lambda, ensuring scalability without server management. Data storage and connection tracking are managed using Amazon DynamoDB. User authentication is implemented using Amazon Cognito for secure access. The development follows an Agile approach with iterative development, testing, and continuous deployment.

Services used in this Project:

- Amazon API Gateway – manages real-time WebSocket connections
- AWS Lambda – handles backend logic and message processing
- Amazon DynamoDB – stores connection IDs and chat data
- Amazon Cognito – provides user authentication and security

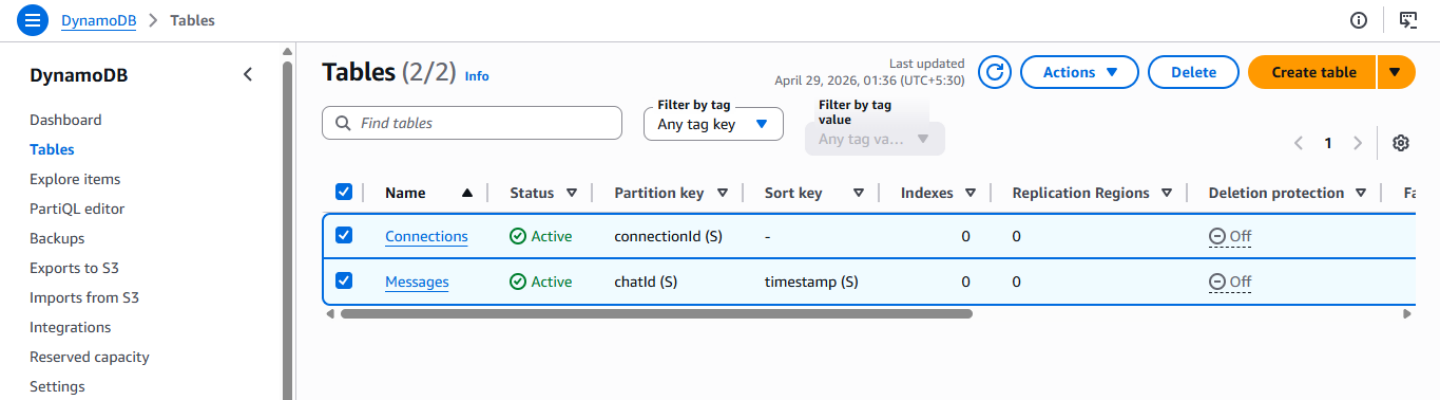
Architecture Diagram:



Implementation and Execution of Real-Time Chat App using WebSockets and Serverless

STEP 1: Create DynamoDB Tables

1. Connections Table
2. Messages Table

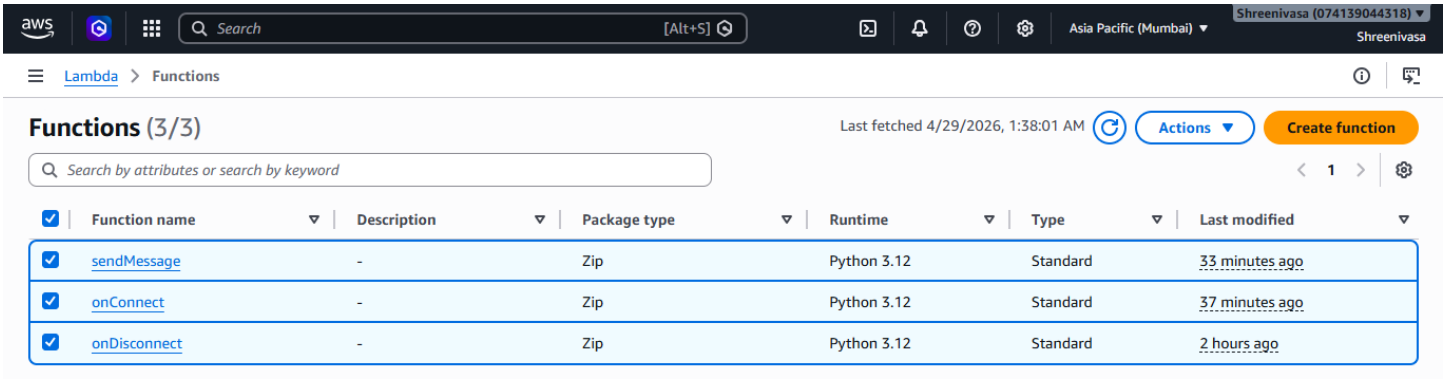


STEP 2: Create Lambda Functions

Go to AWS Lambda → Create function

Created 3 functions:

1. onConnect
2. onDisconnect
3. sendMessage



STEP 3: Create WebSocket API

Go to API Gateway → Create API → **WebSocket**

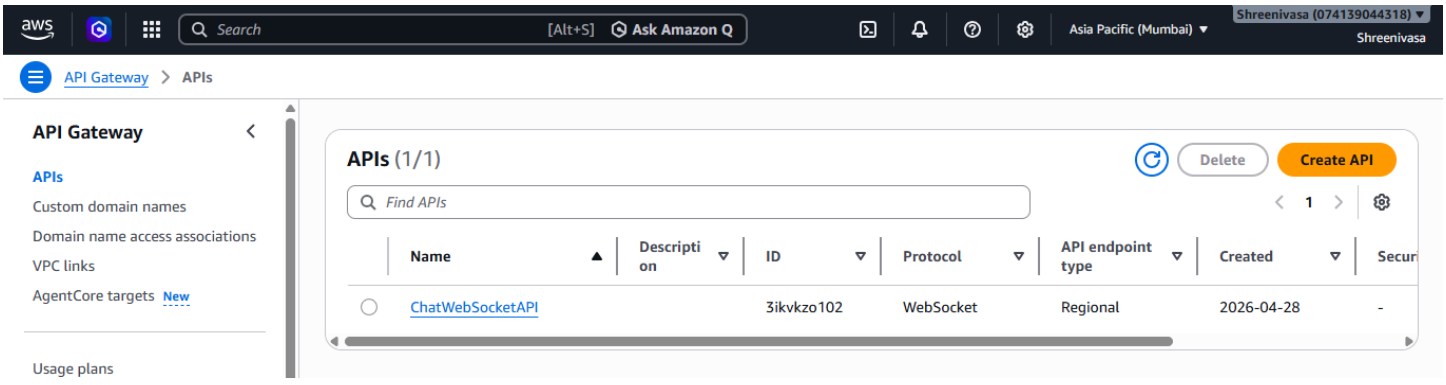
Name: ChatWebSocketApi

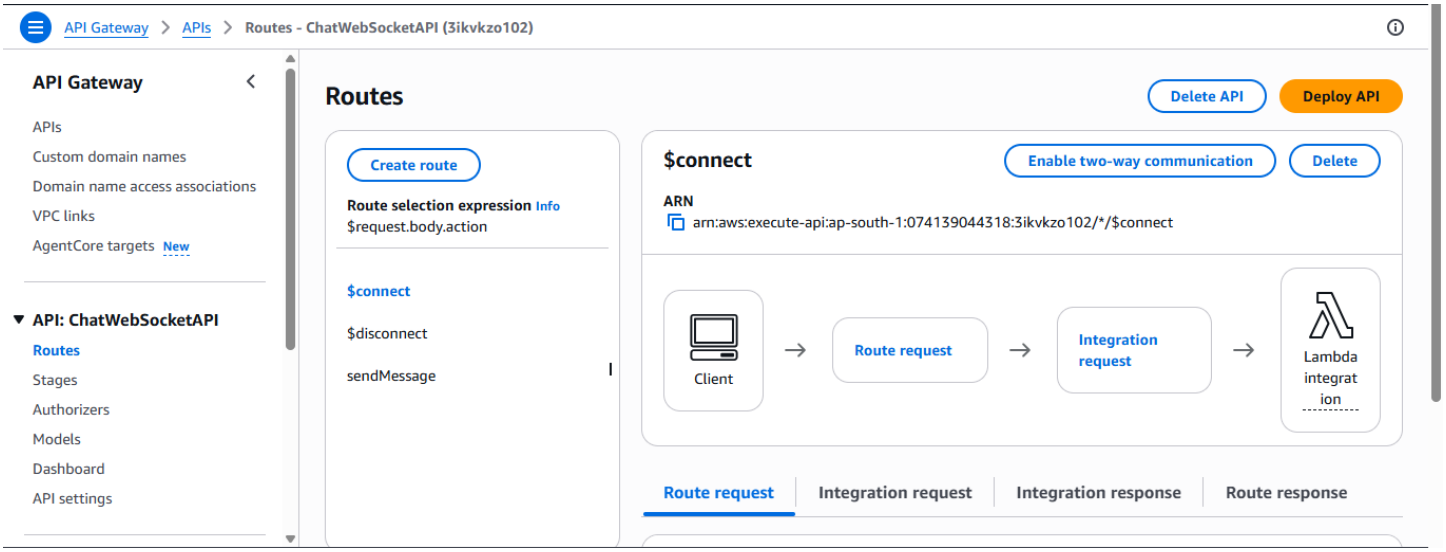
Configure Router names:

$connect

$disconnect

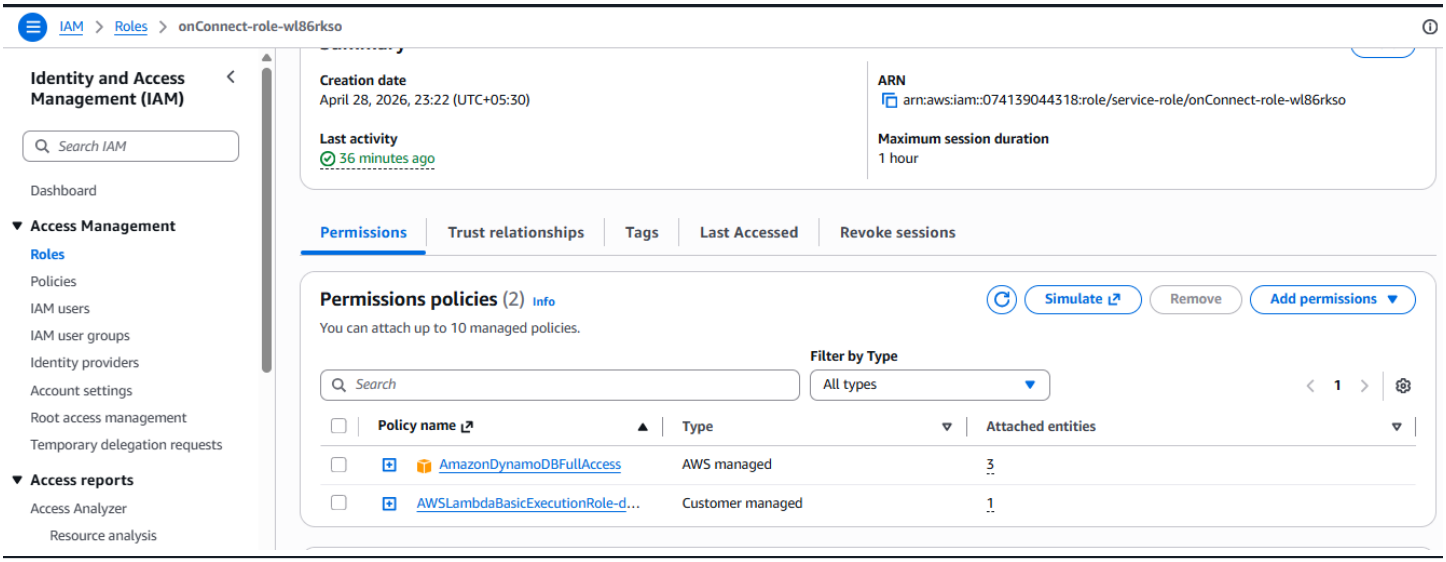
sendMessage

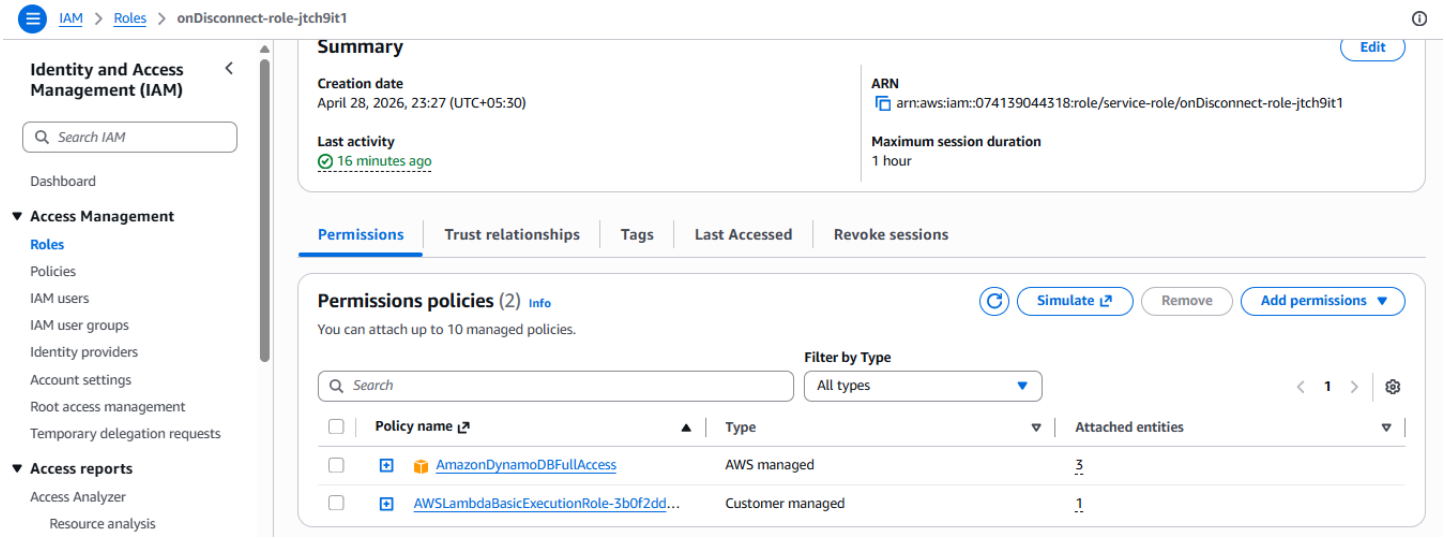


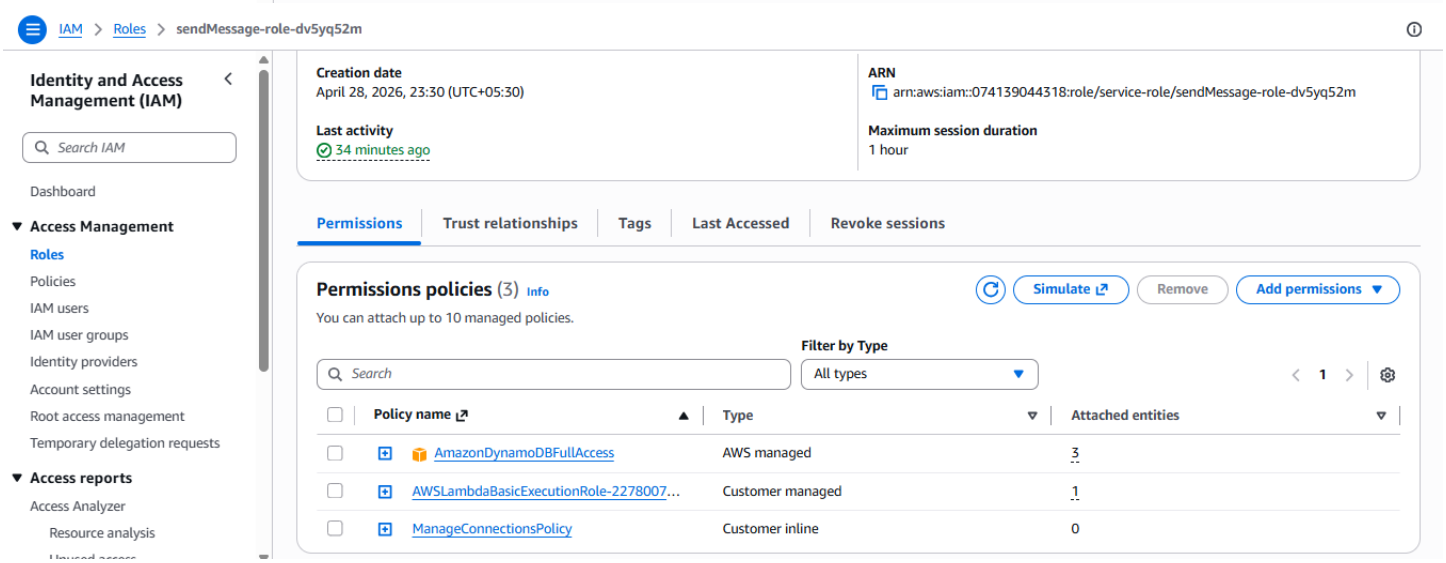


STEP 4: Set IAM Permissions for all the functions

Role: AmazonDynamoDBFullAccess

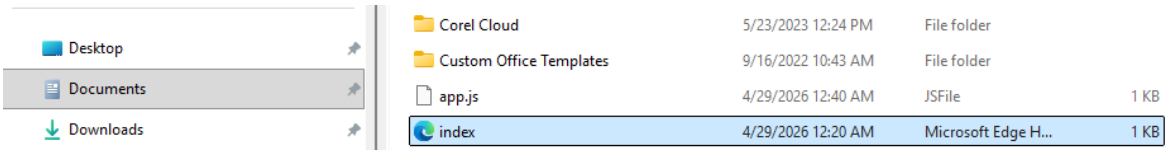






STEP 6: Frontend (Simple HTML + JS)

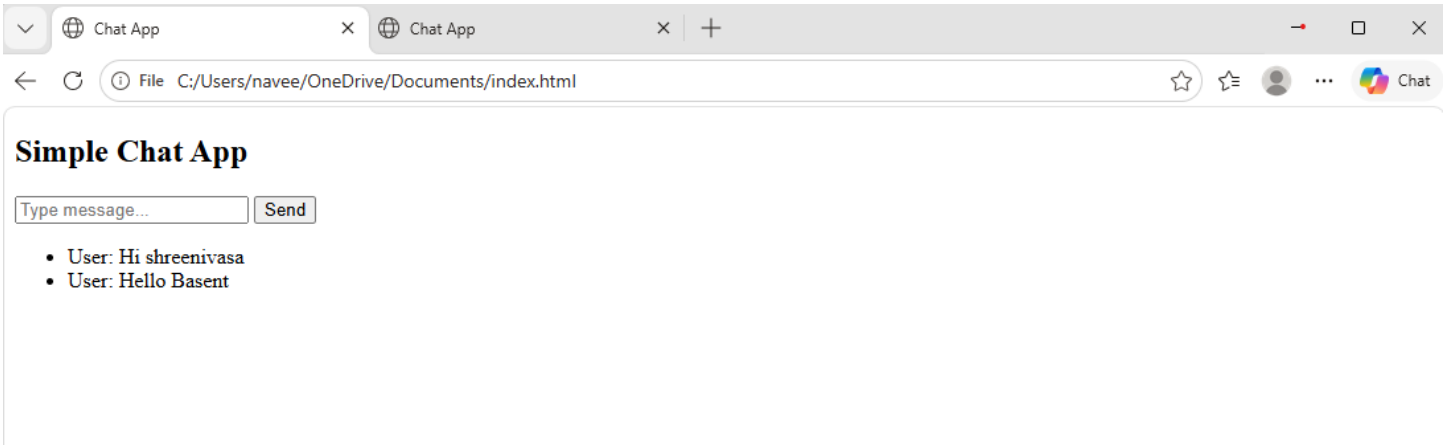
Created **index.html** and **app.js** files with the script I the local



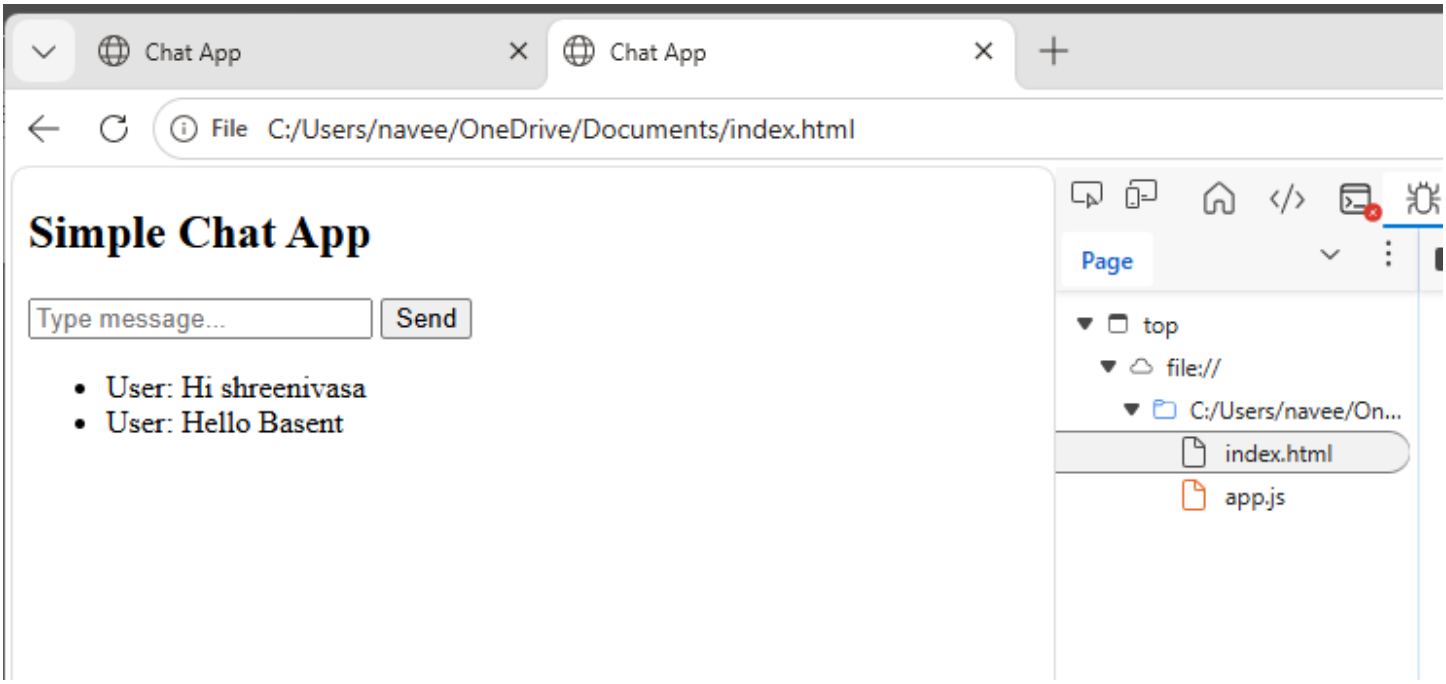
STEP 7: Deploy & Test

1. Open browser
2. Connect WebSocket
3. Send messages
4. Open multiple tabs → test real-time chat

Tab 1:



Tab 2:



Conclusion:

In conclusion, the Real-Time Chat App using WebSockets and Serverless is a scalable and efficient communication system that enables instant messaging between users without managing servers. It leverages Amazon API Gateway for real-time connectivity and AWS Lambda for backend processing. Data handling is managed through Amazon DynamoDB, ensuring fast and reliable storage. User authentication is secured using Amazon Cognito, making the application safe and scalable. Overall, this project demonstrates strong skills in cloud computing, real-time systems, and serverless architecture.